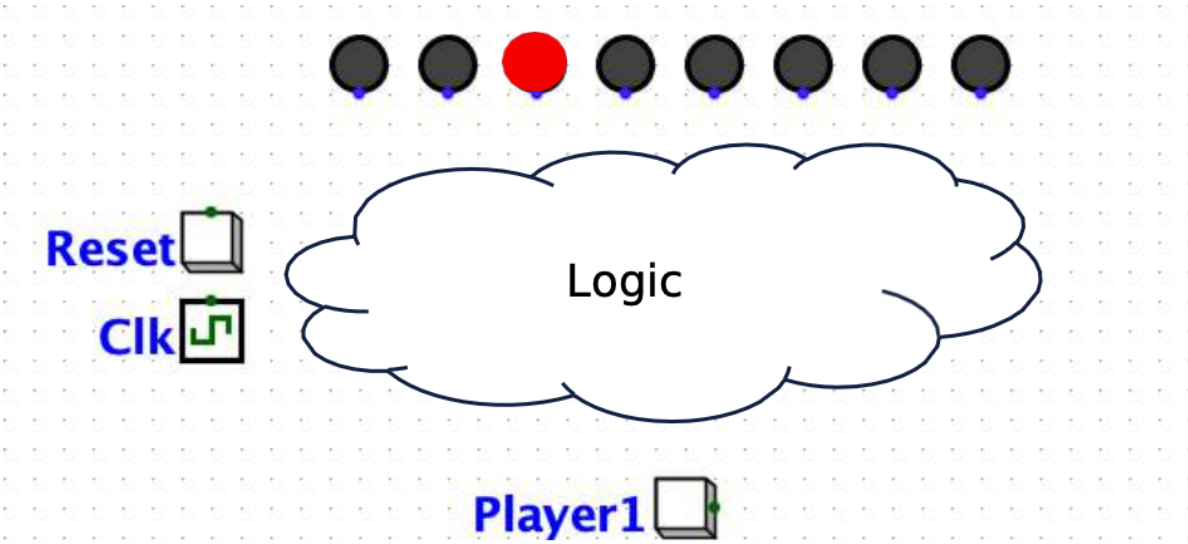
National Technical University
of Athens
Digital Systems Lab
ECE NTUA 2025-26

Implementation of Control Block, Lab 6 Report

ECE NTUA 2025-26

Digital Systems Laboratory



Due to 31/5

Adam Moudrikah (03125575)
Konstantinos Chatzipapas (03123131)

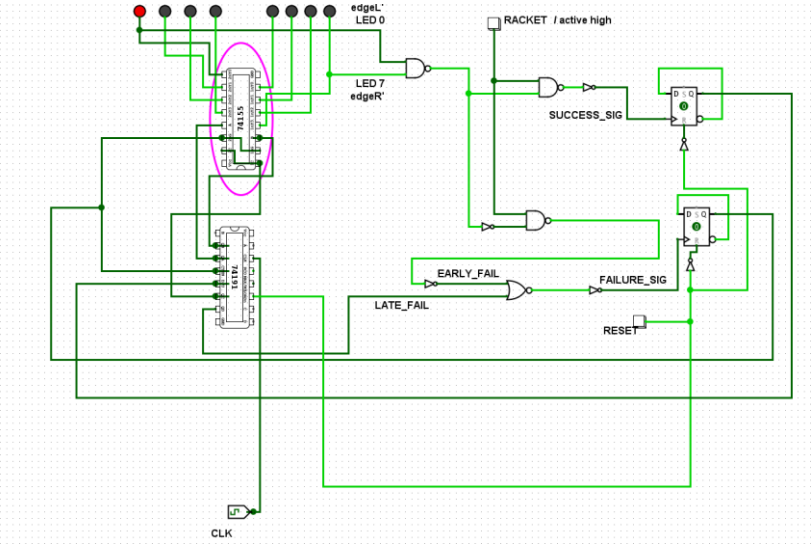
Introduction

In the previous laboratory report #5, we explained the **logic design of the control circuit** for our Pong Project. At a schematic level, this logical subsystem adds **memory** and **proper feedback** to the player's actions. In other words, with the addition of the control block, the circuit no longer simply moves a ball to the right or to the left; instead, it "remembers" the ball's direction and **responds** correctly to the player's **Reset** and **Racket** button presses according to the rules of the game.

In this report, we are required to describe the **implementation** and **integration** stage of the control subcircuit, thereby completing the Pong Game project.

Control Block's Logic Design

The logic design analyzed extensively in Laboratory Report #5 is also used at this stage. Specifically, we implement our circuit using the same **asynchronous** logic configuration, based on **two flip-flops** and **logic gates**. The design developed in the previous stage is recalled below;



The control logic configuration utilizes 3 NAND gates, 1 NOR gate, 6 inverters and 2 D Flip Flops. The final configuration **passes instructors' limitations' guidelines**.

At the logical level, certain aspects require particular attention, including **edge cases** and other logical specificities of our circuit. More specifically;

- The first D flip-flop is responsible for the **correct change of direction**. It is configured as a T, or toggle, flip-flop, since this is the functionality required in the specific game. Its operation is asynchronous with respect to the main system clock: the ***success* signal** is applied as its clock event, and upon this event the **flip-flop toggles the direction state**. Its output **drives the Up/Down** input of the counter. Its *reset* input is driven by an asynchronous *reset* signal originating from an active-low push button, which is inverted through an inverter. The asynchronous *reset* has the highest priority and, in this specific flip-flop, when activated it forces *Q=0* - corresponding to motion from left to right.

- The second D flip-flop is used as a **memory element for the game-failure state**. Through the *Reset* signal, its output Q is initialized to logic 0, a state in which the enable inputs of the integrated circuits allow normal game operation. The $FAILURE$ signal, which is generated by the Early Hit and Late/No Hit conditions, is applied to the clock input of the flip-flop.
  Instead of tying the D input permanently to logic 1, the connection $D=Q'$ was used, so that the D flip-flop operates as a T flip-flop and changes state upon the first failure event. Since, before each new game, *Reset* forces $Q=0$, the first $FAILURE$ pulse drives the output to $Q=1$, disabling the display (; counter and decoder) through the enable signals and placing the system in the game-over state.
  With this modification, additional wiring of a constant logic 1 to the D input is avoided. Subsequently, the circuit disables the display, so a second failure event is not expected before a new *Reset* occurs. For this reason, in our implementation, this configuration is functionally sufficient.

Hardware Implementation

The next challenge is the **transition** from the **logic-level circuit design** to its physical **hardware implementation on perfboard**. The objective here is to describe the additional interconnections required in order to implement the above logic in practice. We have one TTL IC, the 7474, which integrates the two D flip-flops required by our design; one 7400 TTL IC, which contains the three NAND gates used in the circuit; one 7402 TTL IC, from which one NOR gate is used; and one 7404 TTL IC, which provides the

five NOT inverters required. In addition, we have two pull-down switch circuits, as well as capacitors and resistors that will be used later in the debouncing stage.

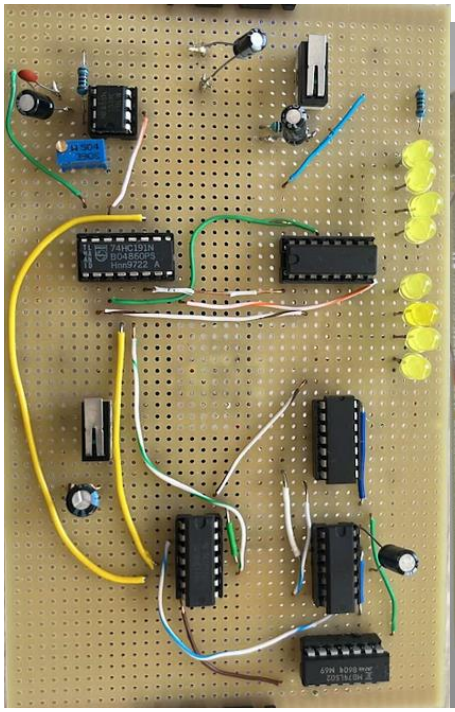


Figure 1; Top view of the perfboard

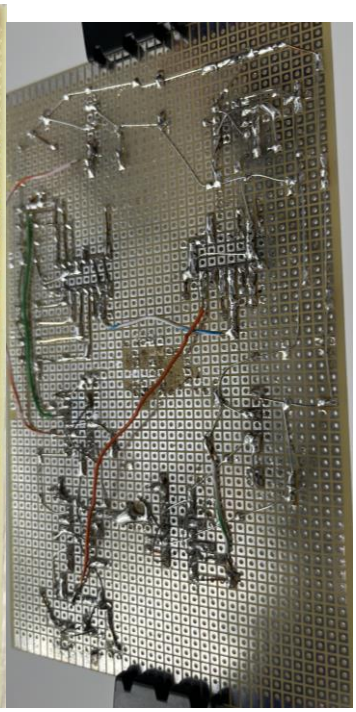


Figure 2; Bottom side view

Additional Integrated Circuits — TTL ICs

7474 / 74LS74 — Dual D Flip-Flop

The 7474 contains two independent D flip-flops, each with D, CLK, PRE, and CLR inputs, and Q, Q' outputs.

7400 / 74LS00 — Quad 2-input NAND

The 7400 contains four independent two-input NAND gates.

7402 / 74LS02 — Quad 2-input NOR

The 7402 contains four independent two-input NOR gates.

7404 / 74LS04 — Hex Inverter

The 7404 contains six independent inverters

The interconnections of the integrated circuits are implemented so as described in the logic design of our circuit, which was analyzed extensively in Report #5.

Momentary Push Button Switches (SPDT)

We use two **momentary push button switches**, responsible for the player's **racket hit** and for **restarting/initializing the game**, respectively. Their structure is characterized by three terminals: NC, NO, and COM/OUT.

For the ***Racket* signal switch**, we use active-high logic. Specifically, NC is connected to GND, NO is connected to VCC, and OUT is connected to the inputs of the two NAND gates. When the switch is not pressed, OUT is connected to NC; therefore, logic 0 is applied at the output, as expected. Conversely, when the switch is pressed, NO is connected to COM, driving the output to logic .

For the ***Reset* switch**, **active-low logic** is used. More specifically, the NC terminal is connected to VC and the NO terminal is connected to GND, so that when the switch is not pressed, $OUT=VCC$, corresponding to logic 1, whereas when the switch is pressed, $OUT=GND$, corresponding to logic 0.

This signal also **drives** the $LOAD$ **input** of the counter, so that when *Reset* is activated, the game is **initialized at position 0**. After passing through an inverter, the same signal is also applied **to the asynchronous reset inputs** of the two flip-flops.

Noise Reduction Practices

Decoupling Capacitors

A standard and long-established practice for minimizing noise-related issues and small signal disturbances in the microelectronic behavior of wires and components is the use of **decoupling capacitors**. More specifically, when a TTL IC switches state—for example, when the output of a logic gate changes or a flip-flop toggles—it **momentarily** r**equires additional current** from the power supply. The power supply and the breadboard/perfboard interconnections cannot always provide this current instantaneously, due to the **parasitic resistance** and **inductance of the wires and conductors**. A **decoupling capacitor mitigates this effect** by locally supplying or absorbing short current pulses, thereby stabilizing the supply voltage near the integrated circuit.

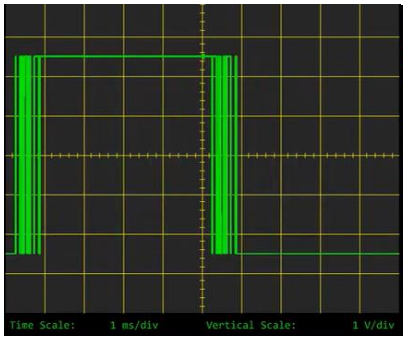
In our circuit, we placed **two** such **decoupling capacitors**, each with a capacitance of $10\,\mu F$. One of them was placed relatively **close to the NOT gate**, since this gate ultimately generates the critical success or failure signal that is applied to the flip-flops. Therefore, we considered noise minimization at this point to be particularly important. This design choice was **experimentally validated**, as the behavior of the circuit became more stable after the addition of the capacitor.

Debouncing Capacitors

Furthermore, since in the **asynchronous implementation** the flip-flops are triggered by **logic-derived event pulses** rather than by a single, stable, well-defined system clock, additional **noise suppression** in the **control block** is essential. This ensures that the signals reaching the flip-flops are clean and logically valid. The main focus is therefore placed on the *Racket* and *Reset* switches, since insufficient noise reduction at these inputs could cause the flip-flops to receive erroneous clock pulses.

Several **switch-debouncing techniques** can be applied to this type of circuit. Most of them address the fundamental problem described below.

When a mechanical switch is pressed, its metallic contacts do not make a single clean transition. Instead, at a microscopic level, they perform **several rapid successive contacts** and separations, which are not perceptible to the user. As a result, the voltage produced at the switch output **may alternate several times between logic 0 and logic 1**, producing the waveform shown in the corresponding figure.



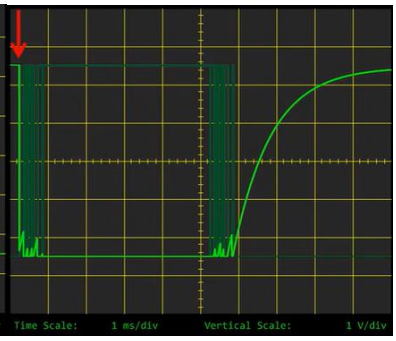


Figure 2; Problematic switch behavior *Figure 3; Improved behavior due to capacitor*

The objective of the debouncing circuit is to introduce “inertia” into the electrical response of the switch output, so that these **rapid voltage transitions are not transferred directly to the digital logic**. This is achieved by connecting a capacitor

between the voltage output node and ground. Due to the electrical properties of the capacitor, the voltage across its terminals cannot change instantaneously; a rapid voltage variation would require **a large transient current**. Consequently, the output voltage changes more gradually, and **short-duration bounce pulses are attenuated before they can be interpreted as valid logic transitions.**

In practice, the digital input changes state only when the filtered voltage crosses the corresponding logic threshold. Therefore, brief unwanted bounces that do not drive the voltage beyond the required threshold are **effectively rejected by the circuit**.

Another important practice implemented in our design was the connection of all unused TTL input pins to defined default logic levels, in order to **prevent them from remaining floating and causing unpredictable behavior** .